LAPORAN UTS

PREDIKSI BIAYA MEDIS ASURANSI MENGGUNAKAN DEEP NEURAL NETWORK

PEMBELAJARAN MENDALAM

Oleh:

Kelompok: 6

Anggota:

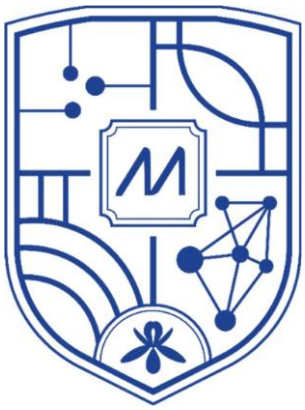
Delarosa Butar Butar (231111254)

Ryan Peace Simbolon (231111190)

Grace Debora Sitompul (231111836)

Saskia Anggi Pratiwi (231110909)

Boris Sanjaya Siregar (231111783)



SEMESTER GANJIL

TA. 2026-2027

DAFTAR ISI

DAFTAR ISI **2**
DAFTAR GAMBAR **4**
DAFTAR TABEL **5**
PEMBAGIAN TUGAS **6**
BAGIAN A. LAPORAN TEKNIS PROYEK **7**
1. KOMPLEKSITAS MASALAH **7**
1.1 Rumusan dan tujuan proyek **7**
1.2 Mengapa permasalahan cukup kompleks **7**
1.3 Tantangan utama **7**
1.4 Hubungan dengan Tugas Proyek 2 ..... **7**
2. *DATASET* YANG DIGUNAKAN **8**
2.1 Sumber dan status *dataset* **8**
2.2 Struktur data, tipe dan variabel target **8**
2.3 Ukuran, karakteristik dan kompleksitas **9**
3. *EXPLORATORY DATA ANALYSIS* (EDA) **9**
3.1 Distribusi label, kategori dan sampel **9**
3.2 Histogram dan transformasi target **10**
3.3 Perbedaan biaya menurut status merokok **10**
3.4 Hubungan prediktor dengan target dan korelasi **11**
3.5 Pemeriksaan kebocoran dan temuan EDA **11**
4. *PREPROCESSING* DATA **12**
4.1 Langkah pemrosesan **12**
4.2 Pembagian *train / validation / test* **12**
4.3 Delapan fitur akhir **13**
5. MODEL YANG DIGUNAKAN **13**
5.1 Metode dan alasan pemilihan **13**
5.2 Arsitektur DNN dan parameter **13**
5.3 Konfigurasi pelatihan **13**
5.4 Dua fitur unik yang benar-benar ada pada kode **14**
6. HASIL EVALUASI MODEL **14**
6.1 Metrik dan dasar penilaian **14**

6.2 Perbandingan delapan eksperimen pada *validation set* **15**
6.3 Evaluasi akhir pada data *testing* **15**
6.4 Kurva *loss* dan diagnosis kesalahan **16**
6.5 *Overfitting*, *underfitting*, dan batas evaluasi **17**
7. TEKNOLOGI YANG DIGUNAKAN **17**
8. *DEPLOYMENT* DAN DEMO **18**
8.1 Artefak aplikasi dan tautan **18**
8.2 Tahapan *deployment* berdasarkan repository GitHub **18**
8.3 Alur inferensi dan contoh *input-output* **19**
8.4 Demonstrasi aplikasi dan keterkaitan hasil eksperimen **20**
8.5 Dokumentasi visual tahapan publikasi **23**
9. LAMPIRAN **28**
9.1 Pembersihan, *encoding* dan standardisasi **28**
9.2 Arsitektur, pelatihan dan seleksi DNN **29**
9.3 Penyimpanan model dan inferensi aplikasi **32**
9.4 Evaluasi, visualisasi dan konsistensi hasil **35**
9.5 Analisis tambahan dampak data leakage (ablasi independen) **37**
10. DAFTAR PUSTAKA **41**
BAGIAN B. TAUTAN **42**
TAUTAN PROYEK **42**
1. *Source code* (GitHub) **42**
2. Video demo dan tahapan *deployment* **42**
3. Aplikasi yang sudah di-deploy **42**

DAFTAR GAMBAR

Gambar 1. Distribusi biaya, transformasi *log1p*, dan status merokok 10
Gambar 2. Sebaran biaya medis berdasarkan status merokok 10
Gambar 3. Hubungan usia, *BMI*, dan rerata biaya menurut wilayah 11
Gambar 4. Matriks korelasi fitur numerik dari *notebook* 11
Gambar 5. Diagram arsitektur DNN terbaik berdasarkan kode *notebook* 13
Gambar 6. Kurva *loss* model DNN awal 16
Gambar 7. Prediksi versus aktual dan sebaran residual DNN pada data uji 16
Gambar 8. Perbandingan MAE dan RMSE tiga model pada data uji 17
Gambar 9. Alur inferensi aplikasi dari kode implementasi 19
Gambar 10. Bukti aplikasi Streamlit: masukan, hasil prediksi, dan informasi model 20
Gambar 11. Contoh prediksi DNN dengan galat kecil pada aplikasi Streamlit 21
Gambar 12. Contoh prediksi DNN dengan galat besar pada aplikasi Streamlit 22
Gambar 13. Tampilan repository GitHub dan struktur berkas proyek 23
Gambar 14. Tampilan konfigurasi publikasi aplikasi pada Streamlit 24
Gambar 15. Tampilan ringkasan status penerbitan dan log aplikasi 25
Gambar 16. Tampilan formulir masukan dan hasil estimasi biaya medis 26
Gambar 17. Tampilan halaman informasi metode dan hasil evaluasi 27

DAFTAR TABEL

Tabel 1. Pembagian tugas anggota kelompok 6
Tabel 2. Struktur dan peran atribut *dataset* 8
Tabel 3. Statistik ringkas *dataset*..... 9
Tabel 4. Distribusi kelompok kategorikal (data awal) 9
Tabel 5. Pemeriksaan kebocoran target (data awal) 11
Tabel 6. Konfigurasi pelatihan model 13
Tabel 7. Penilaian bukti dua fitur unik 14
Tabel 8. Delapan skenario eksperimen arsitektur DNN (validasi)..... 15
Tabel 9. Hasil pengujian final tiga model..... 15
Tabel 10. Versi teknologi dan komponen proyek..... 17
Tabel 11. Status bukti *deployment* serta demo 20
Tabel 12. Perbandingan ablasi dengan dan tanpa kolom premium (data uji)..... 37

PEMBAGIAN TUGAS

Tabel 1 menjelaskan pembagian tanggung jawab kelompok. Pengembangan kode dilaksanakan oleh Ryan dan Boris, sementara Grace menangani eksperimen DNN dan analisis hasil. Tugas dataset dan EDA serta pengujian dan video demo dialokasikan kepada Delarosa dan Saskia. Pembagian ini tidak dimaksudkan sebagai audit kontribusi berdasarkan riwayat commit.

Tabel 1. Pembagian tugas anggota kelompok

NIM	Nama	Peran	Rincian pekerjaan
231111254	Delarosa Butar Butar	*Dataset dan EDA*	Menyiapkan pembahasan dataset, pembersihan data, analisis eksploratif, dan grafik.
231111190	Ryan Peace Simbolon	Pengembangan kode	Bersama Boris mengembangkan kode proyek, termasuk pemodelan dan aplikasi.
231111836	Grace Debora Sitompul	Eksperimen DNN dan laporan	Membahas arsitektur dan delapan eksperimen DNN serta menganalisis hasil pada laporan.
231110909	Saskia Anggi Pratiwi	Pengujian dan video demo	Menyiapkan pengujian aplikasi, dokumentasi hasil, dan video demonstrasi.
231111783	Boris Sanjaya Siregar	Pengembangan kode dan deployment	Bersama Ryan mengembangkan kode proyek, integrasi aplikasi, dan proses deployment.

BAGIAN A. LAPORAN TEKNIS PROYEK

1. KOMPLEKSITAS MASALAH

1.1 Rumusan dan tujuan proyek

Penelitian mini ini membangun model regresi untuk mengestimasi *expenses* (biaya medis) dari enam variabel penjelas pada *dataset* asuransi: usia, jenis kelamin, indeks massa tubuh (*BMI*), jumlah anak, status merokok, dan wilayah. Tujuan teknisnya adalah menyusun alur *preprocessing* yang aman dari kebocoran informasi, melatih kandidat *Deep Neural Network* (DNN), dan membandingkan kinerjanya dengan *Linear Regression* serta *Random Forest* pada data uji yang tidak dipakai untuk pemilihan arsitektur.

1.2 Mengapa permasalahan cukup kompleks

- **Hubungan tidak linear.** Usia, *BMI*, status merokok, dan wilayah dapat berkaitan dengan biaya medis secara berbeda sehingga hubungan antarvariabel tidak dapat diasumsikan selalu linear.
- **Target sangat miring.** Nilai biaya medis memiliki ekor kanan panjang dan sejumlah biaya tinggi sehingga pemodelan pada skala asli lebih sulit.
- **Ukuran *dataset* terbatas.** Hanya 1.337 baris setelah pembersihan; DNN besar dapat berisiko mempelajari *noise* dibandingkan pola yang dapat digeneralisasi.
- **Kebocoran target.** Kolom *premium* merupakan persentase dari *expenses*; pemakaiannya sebagai *input* akan memberikan informasi turunan target pada model.
- **Evaluasi regresi.** Performa harus dinilai pada skala biaya asli, bukan hanya pada target yang telah ditransformasi logaritmik.

1.3 Tantangan utama

Tantangan inti meliputi mencegah *data leakage*, menangani target yang miring, memilih konfigurasi DNN untuk sampel kecil, mengevaluasi galat pada biaya tinggi, dan mempertahankan konsistensi antara model yang disimpan, kode aplikasi, README, dan laporan.

1.4 Hubungan dengan Tugas Proyek 2

Pada proyek ini, tidak tersedia hasil Tugas Proyek 2 dengan permasalahan dan dataset yang sepadan untuk dibandingkan secara langsung. Oleh karena itu, bagian ini tidak menyajikan perbandingan sebelum dan sesudah ataupun menyatakan adanya peningkatan

dari Tugas Proyek 2. Cakupan pembahasan difokuskan pada proyek prediksi biaya medis asuransi, meliputi analisis dan prapemrosesan data, pengecualian kolom *premium* untuk mencegah kebocoran target, pengujian delapan konfigurasi DNN, perbandingan dengan *Linear Regression* dan *Random Forest*, serta penerapan inferensi melalui aplikasi *Streamlit*.

2. *DATASET* YANG DIGUNAKAN

2.1 Sumber dan status *dataset*

Sumber eksperimen adalah data/medical_insurance.csv pada repository GitHub proyek [2] (1.338 baris, delapan kolom). Penelusuran *dataset* publik mengidentifikasi kesesuaian dengan Medical Cost Personal Datasets di Kaggle [1] dan salinan insurance.csv [7]. Seluruh 1.338 baris pada tujuh atribut dasar cocok setelah pemetaan sex → gender, smoker → discount_eligibility, charges → *expenses*, serta pembulatan *bmi* dan *expenses*. Kolom *premium* merupakan atribut tambahan yang dihitung dari *expenses*. Repository GitHub mendokumentasikan berkas CSV yang digunakan dalam eksperimen, tetapi tidak memuat bukti siapa yang pertama kali memperoleh dataset atau alamat unduhan yang digunakan. Karena itu, sumber publik yang cocok dapat diidentifikasi, sedangkan riwayat perolehan data oleh anggota tidak dapat diverifikasi dari repository.

2.2 Struktur data, tipe dan variabel target

CSV memiliki 1.338 observasi dan delapan kolom sebelum dibersihkan. Enam kolom digunakan sebagai kandidat masukan, satu kolom menjadi target regresi, dan satu kolom dikeluarkan karena kebocoran target. Setelah satu baris duplikat dihapus, jumlah observasi menjadi 1.337.

Tabel 2. Struktur dan peran atribut *dataset*

Atribut	Tipe / contoh	Peran
age	Numerik: usia	Masukan
gender	Kategorikal: *female, male*	Masukan
bmi	Numerik: indeks massa tubuh	Masukan
children	Numerik: 0-5	Masukan
discount_eligibility	Kategorikal: yes=merokok, no=tidak merokok	Masukan
region	Kategorikal: 4 wilayah	Masukan
expenses	Numerik kontinu	Target regresi
premium	Numerik kontinu	Dikeluarkan: turunan target

2.3 Ukuran, karakteristik dan kompleksitas

Tabel 3. Statistik ringkas *dataset*

Komponen	Nilai berdasarkan berkas / *output* tersimpan
Baris awal	1.338
Kolom awal	8
Baris setelah duplikat dibuang	1.337
Missing value	0
Baris duplikat	1
Usia (min, rerata, maks)	18 ; 39,207 ; 64
BMI (min, rerata, maks)	16,0 ; 30,665 ; 53,1
Biaya (min, median, rerata, maks)	1.121,87 ; 9.382,03 ; 13.270,42 ; 63.770,43

Dataset lokal merupakan turunan dari data publik biaya asuransi [1], [7], bukan data klinis primer yang telah diverifikasi. Kolom discount_eligibility merupakan nama teknis lama untuk variabel smoker pada sumber publik: yes berarti merokok dan no berarti tidak merokok; kolom ini bukan pengukuran kelayakan diskon. Kolom *premium* dihitung dari *expenses* dan dikeluarkan dari prediktor. Nilai *bmi* dan *expenses* telah dibulatkan untuk berkas proyek. Nama kolom teknis dan bobot model tetap dipertahankan untuk kompatibilitas; istilah pada grafik lama dibaca sebagai status merokok.

3. *EXPLORATORY DATA ANALYSIS* (EDA)

3.1 Distribusi label, kategori dan sampel

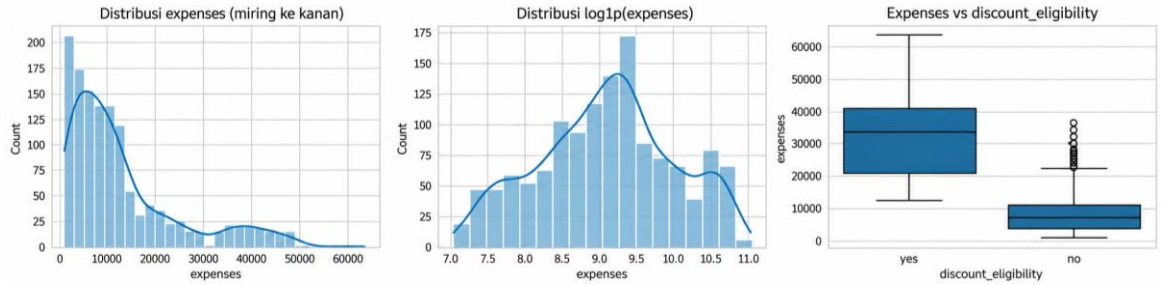
Proyek ini adalah regresi, sehingga tidak memiliki kelas/label diskret untuk dihitung seperti klasifikasi. Distribusi target dipelajari melalui histogram, sedangkan kelompok kategori dianalisis sebagai calon prediktor. Frekuensi pada Tabel 4 berasal dari 1.338 baris data awal sebelum penghapusan satu duplikat.

Tabel 4. Distribusi kelompok kategorikal (data awal)

Kolom	Kelompok / jumlah
gender	*male*: 676; *female*: 662
discount_eligibility	*tidak merokok (no): 1.064; merokok (yes): 274*
region	*southeast*: 364; *southwest*: 325; *northwest*: 325; *northeast*: 324
children	0: 574; 1: 324; 2: 240; 3: 157; 4: 25; 5: 18

Contoh baris pertama pada *dataset*: *age*=19, *gender=female*, *bmi*=27,9, *children*=0, *discount_eligibility=yes (merokok)*, *region=southwest*, *expenses*=16.884,92, dan *premium*=168,8492. Data identitas individu tidak tersedia.

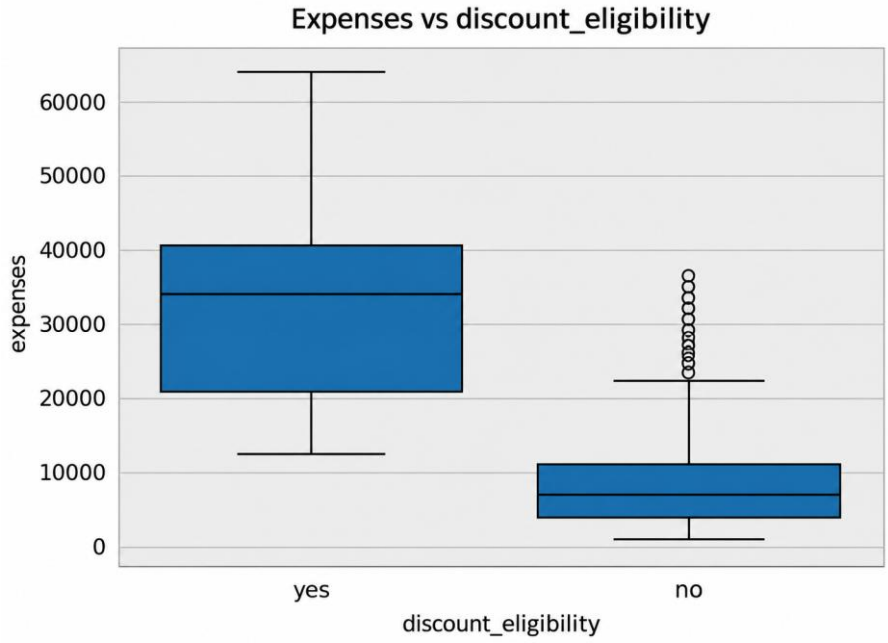
3.2 Histogram dan transformasi target



Gambar 1. Distribusi biaya, transformasi *log1p*, dan status merokok

Gambar 1 menunjukkan distribusi *expenses* miring ke kanan. Distribusi *log1p(expenses)* lebih terkompresi; karena itu *notebook* menggunakan transformasi pada target saat pelatihan, lalu mengembalikan prediksi menggunakan *expm1*. Ketiga panel tersebut merupakan *output* asli *notebook*, bukan hasil hitung ulang. Nama discount_eligibility pada panel ketiga adalah label teknis lama yang berasal dari variabel smoker.

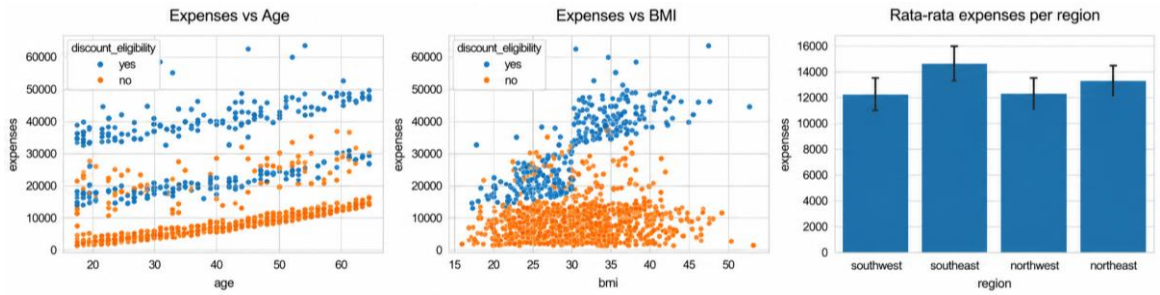
3.3 Perbedaan biaya menurut status merokok



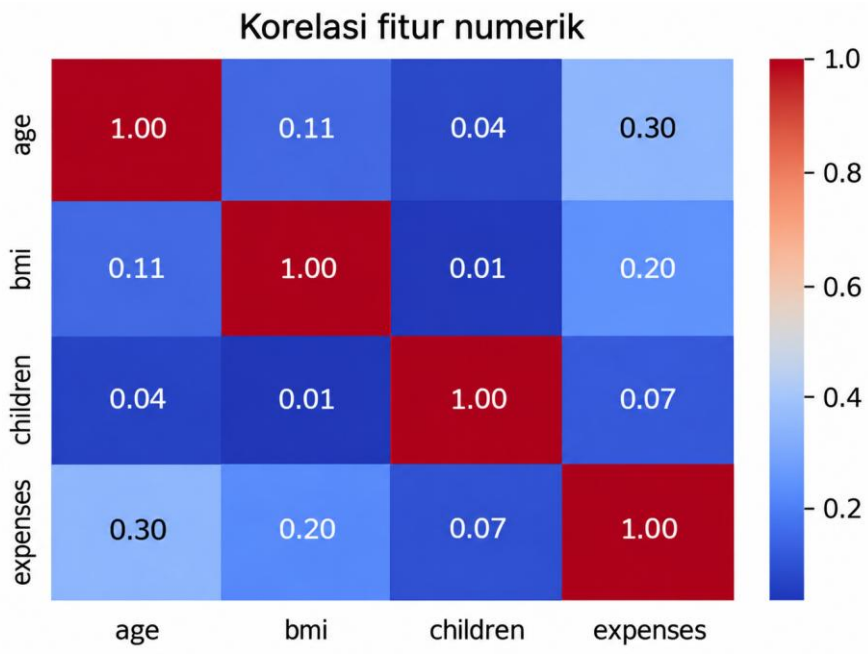
Gambar 2. Sebaran biaya medis berdasarkan status merokok

Kelompok merokok (discount_eligibility=yes) memiliki median biaya sekitar 34.456,35, lebih tinggi daripada kelompok tidak merokok (no) dengan median sekitar 7.345,40 (ringkasan deskriptif CSV). Perbedaan tersebut menunjukkan asosiasi pada data, bukan bukti bahwa status merokok menyebabkan biaya medis berubah.

3.4 Hubungan prediktor dengan target dan korelasi



Gambar 3. Hubungan usia, *BMI*, dan rerata biaya menurut wilayah



Gambar 4. Matriks korelasi fitur numerik dari *notebook*

Pola pada *scatter plot* tidak sepenuhnya linear dan memperlihatkan variasi biaya antar kelompok status merokok. Matriks korelasi yang ditampilkan berasal dari kolom numerik *age*, *bmi*, *children*, dan *expenses*; korelasi tidak diinterpretasikan sebagai hubungan kausal.

3.5 Pemeriksaan kebocoran dan temuan EDA

Tabel 5. Pemeriksaan kebocoran target (data awal)

Rasio *premium* / *expenses*	**Jumlah observasi**
0,01 (1%)	444
0,02 (2%)	803
0,04 (4%)	91
Korelasi *premium* vs *expenses*	0,854 (*output notebook*)
Jumlah dengan rasio turunan target yang tepat	1.338 / 1.338 (100%)

Pemeriksaan presisi desimal terhadap 1.338 observasi awal menunjukkan bahwa seluruh baris (100%) memenuhi hubungan *premium* = 0,01 × *expenses*, 0,02 × *expenses*, atau 0,04 × *expenses*. Sebanyak 444 baris memiliki rasio 1%, 803 baris 2%, dan 91 baris 4%. Informasi ini merupakan kebocoran target karena *premium* dihitung dari *expenses*. Dampak terhadap metrik MAE dan RMSE kini diperiksa melalui eksperimen ablasi independen pada dua model *baseline*, bukan dengan menjalankan ulang *notebook* atau model DNN; uraian dan angka ada pada Lampiran 9.5 dan Tabel 12.

4. *PREPROCESSING* DATA

4.1 Langkah pemrosesan

- **Pemeriksaan data.** *Notebook* mengidentifikasi nol missing value dan satu duplikat; duplikat dihapus dengan *drop_duplicates()*.
- **Pemilihan fitur aman.** Kolom *premium* dikeluarkan; target *expenses* dipisahkan dari fitur masukan.
- **Encoding.** Fitur *gender*, *discount_eligibility*, dan *region* diubah dengan *one-hot encoding*, *drop_first=True*; hasil akhir delapan fitur numerik.
- **Transformasi target.** *log1p* dipakai untuk pelatihan. Pada evaluasi, keluaran dikembalikan ke skala aslinya menggunakan *expm1*.
- **Standardisasi.** *StandardScaler* hanya di-fit pada subset *train*, kemudian dipakai mentransformasi kolom *age*, *bmi*, dan *children* di subset lain.
- **Augmentasi.** Tidak ada augmentasi data karena tugas menggunakan data tabular regresi dan *notebook* tidak mengimplementasikan augmentasi.

4.2 Pembagian *train / validation / test*

Dari 1.337 observasi bersih, *notebook* membagi data dengan rasio mendekati 70% / 15% / 15%, menghasilkan 935 *train*, 201 *validation*, dan 201 *test*. Pengacakan menggunakan *random_state=42*. Perbandingan delapan skenario DNN dilakukan pada *validation set*, sedangkan hasil akhir dilaporkan pada *test* set.

Pencegahan kebocoran yang jelas terlihat dalam kode adalah menghapus *premium* dan menyesuaikan *scaler* hanya dari train. Tidak terdapat bukti tertulis pengelompokan berdasarkan individu atau validasi lintas periode; hal ini menjadi batasan interpretasi pada data dunia nyata.

4.3 Delapan fitur akhir

age, *bmi*, *children*, *gender_male*, *discount_eligibility_yes*, *region_northwest*, *region_southeast*, *region_southwest*. Kategori acuan adalah *female*, *no*, dan northeast. Kolom *input* tersebut juga tercantum di objek preprocessor tersimpan.

5. MODEL YANG DIGUNAKAN

5.1 Metode dan alasan pemilihan

Algoritma utama adalah *Deep Neural Network* tipe *multilayer perceptron* (MLP), sesuai data tabular. CNN tidak diperlukan karena fitur tidak berbentuk citra atau urutan spasial. Dua *baseline* ditambahkan untuk menilai keuntungan relatif DNN: *Linear Regression* (hubungan sederhana) dan *Random Forest* (hubungan nonlinear berbasis ensemble pohon).

5.2 Arsitektur DNN dan parameter

Model DNN awal memakai susunan delapan *input* → Dense(64) → *Dropout*(0,1) → Dense(32) → *Dropout*(0,1) → Dense(16) → *Dropout*(0,1) → satu *output* linear, dengan 3.201 parameter menurut *summary()* yang tersimpan. Namun arsitektur yang dipilih pada validasi adalah *Shallow* (16), yakni delapan *input* → Dense(16, ReLU) → satu *output* linear, tanpa *Dropout*, dengan 161 parameter hasil perhitungan struktur yang tercantum di kode.



Gambar 5. Diagram arsitektur DNN terbaik berdasarkan kode *notebook*

5.3 Konfigurasi pelatihan

Tabel 6. Konfigurasi pelatihan model

Komponen	**Konfigurasi yang tertera pada kode**
Optimizer DNN terpilih	Adam; *learning rate* = 0,001
Aktivasi / keluaran	ReLU pada hidden; linear pada *output*
Loss	*Mean Squared Error* pada *log1p*(*expenses*)
Batch size	32
Maksimum *epoch*	800
EarlyStopping	Monitor: val_loss; patience: 25; restore_best_weights=True
Validasi	201 observasi; tidak sama dengan *test*
Baseline Random Forest	n_estimators=300; min_samples_leaf=3; random_state=42

Model DNN awal dengan tiga hidden *layer* berhenti pada 94 *epoch* berdasarkan *output* notebook. Jumlah *epoch* pelatihan ulang model terpilih *Shallow (16)* tidak dicetak sebagai *output* terpisah sehingga tidak dinyatakan di sini.

5.4 Dua fitur unik yang benar-benar ada pada kode

Fitur unik 1: kontrol kebocoran target sebelum pelatihan

Apa dan mengapa: *premium* merupakan turunan deterministik *expenses*. Bukti data: 1.338 dari 1.338 baris awal (100%) memiliki rasio *premium*/*expenses* tepat 1%, 2%, atau 4% (Tabel 5). Implementasi utama: data.drop(columns=["*premium*"]) sebelum membentuk X. Bukti dampak tambahan: eksperimen ablasi independen (Lampiran 9.5, Tabel 12) menunjukkan bahwa Random Forest memperoleh MAE 2.082,56 dan RMSE 4.583,21 tanpa *premium*, tetapi MAE 359,00 dan RMSE 1.523,66 ketika *premium* disertakan. Performa tampak lebih tinggi karena informasi target bocor, bukan karena kemampuan generalisasi yang valid. Eksperimen tambahan ini hanya untuk model *baseline*; *notebook* dan pelatihan DNN asli tidak dijalankan ulang.

Fitur unik 2: pemilihan arsitektur dan regularisasi berbasis validation RMSE

Apa dan mengapa: perbandingan delapan variasi model untuk memilih kapasitas, *Dropout*, *optimizer*, dan aktivasi yang sesuai data kecil. Implementasi: *configs* di *notebook*, validasi terpisah, dan EarlyStopping. Bukti dampak: tiga lapisan 64-32-16 tanpa *Dropout* menghasilkan *validation RMSE* 7.405,933, sedangkan konfigurasi tiga lapisan + *Dropout* 0,1 menghasilkan RMSE 5.519,690, lebih rendah sekitar 25,47%. Perbandingan ini berasal dari run yang tersimpan, bukan eksperimen baru. Besaran beda belum dapat dianggap efek kausal terisolasi karena hasil pelatihan stochastic.

Tabel 7. Penilaian bukti dua fitur unik

Fitur	Bukti tersedia	Batas bukti
Eksklusi *premium* anti-leakage	Rasio deterministik 1.338/1.338 baris; hasil ablasi terpisah pada dua baseline (Tabel 12)	Ablasi bukan eksperimen asli DNN; efek pada DNN belum diukur
Seleksi delapan skenario / *Dropout*	Tabel validasi delapan arsitektur, *EarlyStopping*, kurva *loss*	Tidak ada pengulangan multi-seed dan uji signifikansi

6. HASIL EVALUASI MODEL

6.1 Metrik dan dasar penilaian

Mean Absolute Error (MAE) mengukur rata-rata selisih absolut; *Root Mean Squared Error* (RMSE) menghukum kesalahan besar lebih kuat; dan R^2 menunjukkan proporsi

variasi target yang dijelaskan relatif terhadap *baseline* rata-rata. Seluruh angka dilaporkan pada skala asli *expenses* setelah prediksi invers transformasi log.

6.2 Perbandingan delapan eksperimen pada *validation set*

Tabel 8. Delapan skenario eksperimen arsitektur DNN (validasi)

Arsitektur	MAE	RMSE	R^2	*Epoch*
Shallow (16)	2.989,057	5.183,080	0,843	371
3 *layer* + *Dropout*, SGD	2.800,050	5.244,905	0,839	61
3 *layer* + *Dropout* 0,1	3.003,861	5.519,690	0,822	183
2 *layer* (32-16)	3.233,663	5.595,109	0,817	211
Deep (128-64-32-16) + D0,2	3.035,360	5.804,232	0,803	98
3 *layer* + *Dropout*, RMSprop	3.355,071	5.853,420	0,800	118
3 *layer* (64-32-16)	3.579,636	7.405,933	0,679	134
3 *layer* + *Dropout*, tanh	4.499,881	7.454,930	0,675	215

Pemilihan arsitektur memakai *validation RMSE* sebagai kriteria, bukan MAE minimum; karena itu *Shallow* (16) terpilih walaupun konfigurasi dengan SGD memberikan MAE validasi lebih rendah. Langkah pemilihan ini sesuai dengan keluaran sel *notebook* yang mengurutkan tabel berdasarkan RMSE.

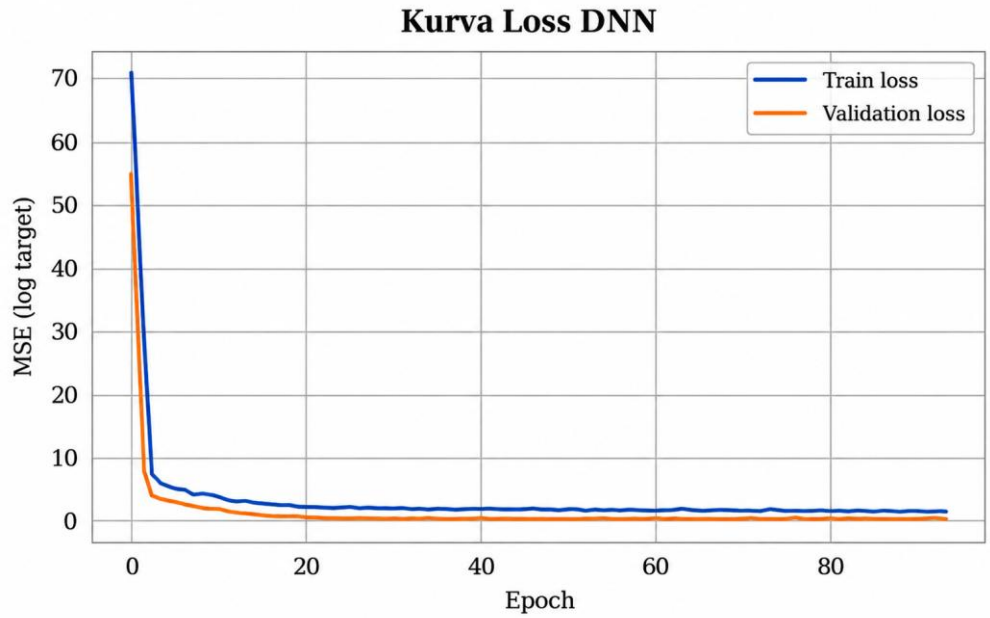
6.3 Evaluasi akhir pada data testing

Tabel 9. Hasil pengujian final tiga model

Model	MAE	RMSE	R^2
Linear Regression	3.570,213	7.230,611	0,694
Random Forest	2.082,563	4.583,214	0,877
DNN (*Shallow* 16)	3.086,642	5.938,255	0,793

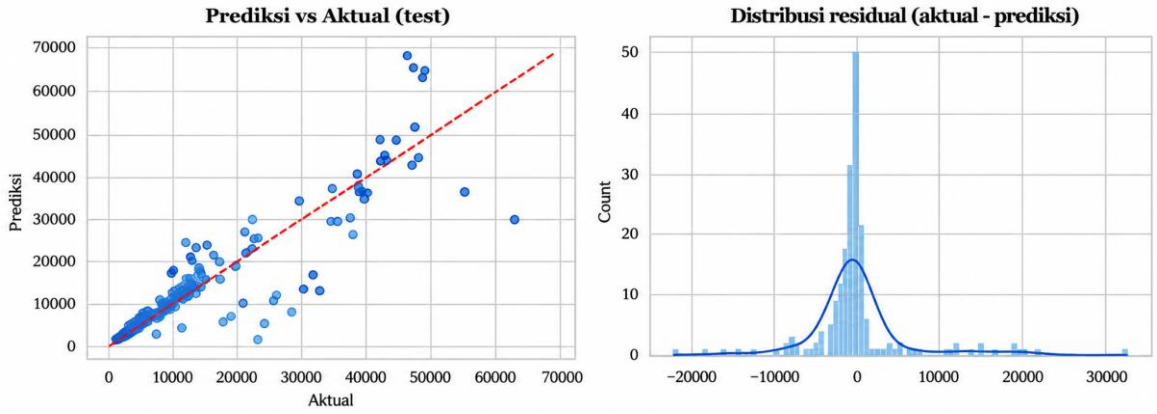
Hasil asli menunjukkan bahwa *Random Forest* terbaik berdasarkan ketiga metrik di atas, DNN mengungguli *Linear Regression*, tetapi belum mampu mengungguli Random Forest. Maka tidak tepat menyatakan DNN merupakan model paling akurat dalam penelitian ini.

6.4 Kurva *loss* dan diagnosis kesalahan



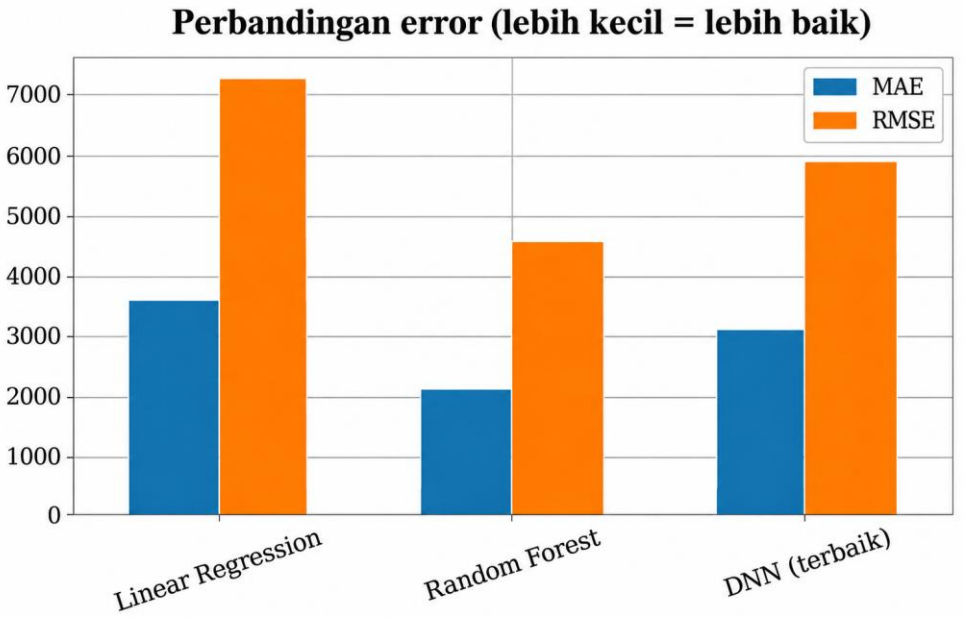
Gambar 6. Kurva *loss* model DNN awal

Grafik kurva *loss* adalah milik model DNN awal yang memiliki susunan 64-32-16 dan *Dropout* 0,1, bukan model final *Shallow* (16). *Notebook* hanya menyimpan visual kurva untuk model awal. Pelaporan ini sengaja memisahkan keduanya agar grafik tidak keliru diberi label sebagai riwayat model final. Metrik accuracy tidak digunakan karena keluaran proyek ini merupakan nilai kontinu (regresi), bukan label klasifikasi.



Gambar 7. Prediksi versus aktual dan sebaran residual DNN pada data uji

Visual memperlihatkan galat tidak seragam: prediksi pada biaya besar dapat menyimpang cukup jauh dari garis ideal. Histogram residual berpusat dekat nol tetapi memiliki ekor, konsisten dengan RMSE DNN yang lebih tinggi daripada MAE. Tidak ada klasifikasi benar/salah seperti pada *dataset* berlabel kelas; contoh kesalahan pada proyek regresi dibahas sebagai selisih aktual dan prediksi.



Gambar 8. Perbandingan MAE dan RMSE tiga model pada data uji

6.5 *Overfitting*, *underfitting*, dan batas evaluasi

Hasil *validation RMSE* model tiga *layer* tanpa *dropout* lebih buruk daripada model dangkal dan varian *dropout*; ini mendukung dugaan bahwa kapasitas berlebih dapat merugikan pada ukuran sampel kecil. Namun diagnosis pasti *overfitting* memerlukan kurva model yang bersangkutan, perbandingan *train-validation* yang lengkap, dan/atau eksperimen pengulangan. Tidak tersedia pengujian prediksi yang benar dan salah dengan label kategorikal karena masalah ini berupa regresi.

7. TEKNOLOGI YANG DIGUNAKAN

Versi berikut diambil dari requirements.txt; informasi *runtime notebook* menyebut TensorFlow 2.21.0, sedangkan README menyatakan penggunaan Python 3.11. Spesifikasi perangkat GPU tidak dicatat pada *output notebook* yang diberikan.

Tabel 10. Versi teknologi dan komponen proyek

Komponen	Versi / fungsi
Python	3.11 (dinyatakan di README)
TensorFlow / Keras	2.21.0 / 3.14.1: pelatihan dan inferensi DNN
scikit-learn	1.6.1: split, *scaler*, *baseline*, metrik
pandas / numpy	2.2.3 / 2.2.5: manipulasi dan numerik
matplotlib / seaborn	3.10.1 / 0.13.2: visualisasi
joblib	1.4.2: serialisasi preprocessor
Streamlit	1.66.1: antarmuka web
Lingkungan eksekusi	*Notebook* Jupyter / IPYNB; informasi GPU tidak terdokumentasi
Penyimpanan model	models/dnn_medical_cost.keras dan models/preprocessor.joblib

Folder aplikasi menggunakan antarmuka Streamlit; *hosting* yang disebut README adalah Streamlit Community Cloud. Pelatihan model dan aplikasi inferensi merupakan dua aktivitas berbeda: aplikasi memuat model tersimpan dan tidak mengeksekusi ulang notebook.

8. *DEPLOYMENT* DAN DEMO

8.1 Artefak aplikasi dan tautan

Alamat repositori proyek adalah https://github.com/boris-sanjaya/Medical_Cost_Prediction_DNN, sedangkan aplikasi Streamlit menggunakan alamat https://medicalcostpredictiondnn-jnbqvke2tgpebqlryh8xgp.streamlit.app/. Struktur proyek terdiri atas app/app.py dengan formulir Status merokok, README.md, requirements.txt, berkas model dan *preprocessor*, *notebook* eksperimen, skrip pengujian, hasil analisis, serta laporan PDF. Pemetaan *input* aplikasi tetap mengikuti delapan fitur dan urutan kolom pada *preprocessor* yang digunakan dalam pelatihan.

8.2 Tahapan *deployment* berdasarkan repository GitHub

- **Menyimpan model.** Sel terakhir *notebook* menyimpan *final* sebagai *.keras* dan scaler/urutan kolom melalui *joblib*. Kedua artefak disimpan pada models/dnn_medical_cost.keras dan models/preprocessor.joblib dalam repository GitHub [2].
- **Menyiapkan inferensi.** Fungsi *predict()* pada app/app.py membentuk delapan kolom dengan urutan yang disimpan, menerapkan *StandardScaler* pada kolom numerik, memanggil *model.predict()*, kemudian *expm1*.
- **Membangun UI.** Formulir dalam app/app.py memuat usia, BMI, jumlah anak, jenis kelamin, status merokok, dan wilayah; tombol Prediksi menampilkan estimasi biaya medis. Nama teknis fitur discount_eligibility_yes dipertahankan karena merupakan bagian dari susunan masukan model dan *preprocessor* tersimpan.
- **Mencatat dependensi.** requirements.txt mencantumkan seluruh library inti dan versinya; README memuat perintah *pip install -r requirements.txt* dan *streamlit run app/app.py*.

- Hosting. Alamat aplikasi *Streamlit* Community Cloud tercantum dalam *README* dan Bagian B. Gambar 13–15 memvisualisasikan alur penyiapan *repository*, konfigurasi publikasi, dan pemeriksaan status aplikasi menurut berkas proyek. Gambar-gambar tersebut tidak dipakai sebagai pengganti riwayat *build* atau *log* asli *dashboard* pengelola.
- **Pengujian.** *Screenshot* menunjukkan aplikasi mengeluarkan estimasi 4.548,49 untuk contoh awal, 9.721,62 untuk kasus galat kecil, dan 30.043,66 untuk kasus galat besar. Dua kasus terakhir dibandingkan dengan nilai aktual data uji. Bukti ini mendukung fungsi inferensi pada contoh yang diuji, bukan pengujian menyeluruh terhadap seluruh konfigurasi server atau *dashboard* hosting.

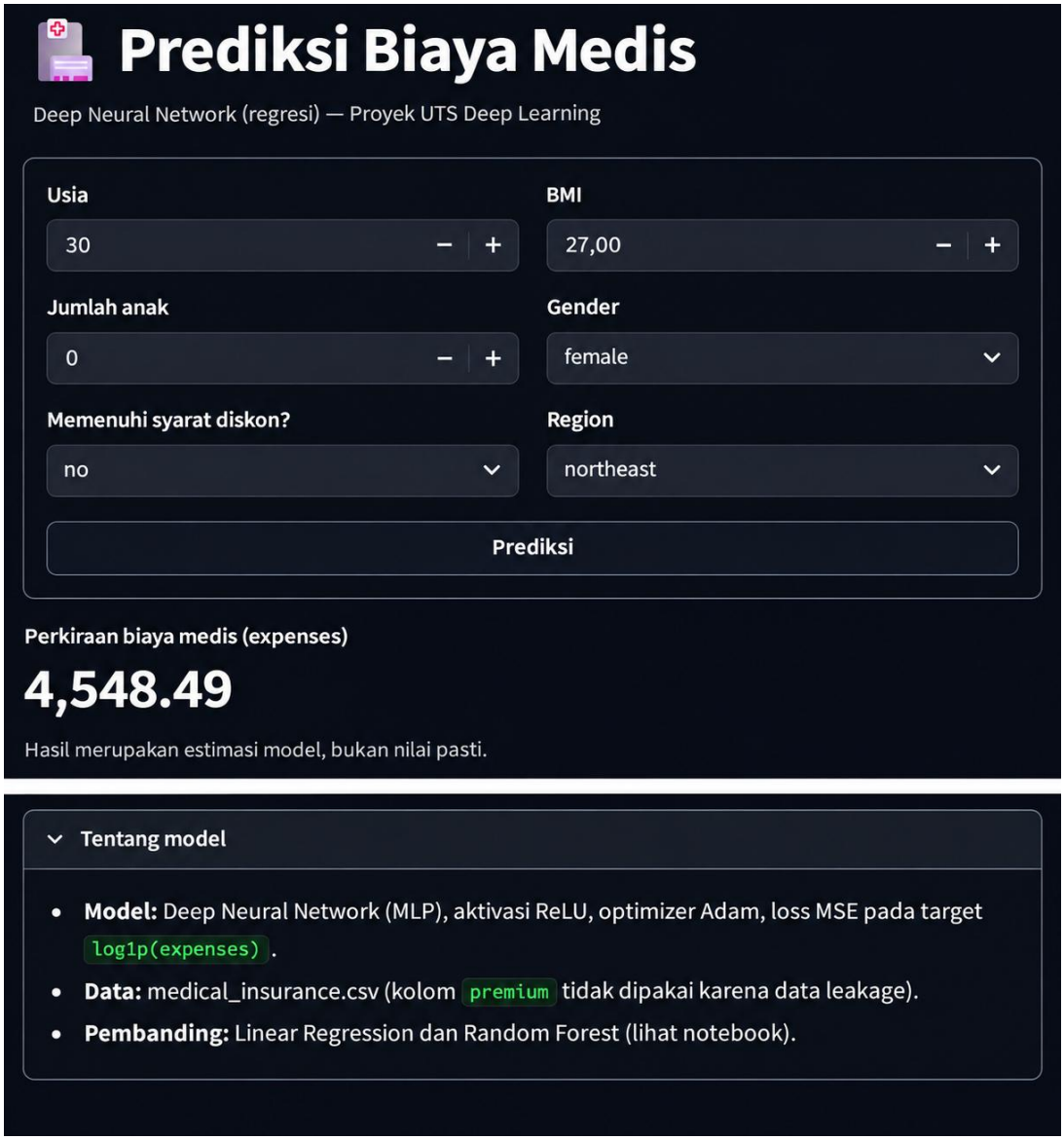
8.3 Alur inferensi dan contoh *input-output*



Gambar 9. Alur inferensi aplikasi dari kode implementasi

Rekaman demo *MP4* pada Bagian B berdurasi 15,51 detik dan memperlihatkan formulir *Streamlit*, panel Tentang model, serta keluaran prediksi 9.721,62 dan 30.043,66. Bagian awal video memperlihatkan masukan usia 30 tahun, BMI 27,00, tanpa anak, perempuan, no, dan northeast. Estimasi 4.548,49 untuk contoh awal didokumentasikan secara terpisah pada Gambar 10. Pilihan no/yes merujuk pada status merokok di *dataset* sumber, bukan kelayakan diskon.

8.4 Demonstrasi aplikasi dan keterkaitan hasil eksperimen



Gambar 10. Bukti aplikasi Streamlit: masukan, hasil prediksi, dan informasi model

Tabel 11. Status bukti *deployment* serta demo

Persyaratan bukti	Status bukti yang tersedia
Model dan preprocessor tersimpan	Ada: .keras dan .joblib
Script inferensi dan UI	Tersedia: app/app.py dengan formulir Status merokok dan inferensi menggunakan model tersimpan.
Daftar dependensi dan *repository*	Tersedia: requirements.txt, README.md, dokumentasi kode, serta laporan PDF pada folder docs/.
Dokumentasi tahapan hosting	Tersedia: penjelasan alur dan gambar pendukung pada Subbab 8.2, Gambar 13–15. *Log* dan waktu *deployment* asli hanya dapat dicek pada *dashboard* pengelola.
Bukti dua kasus prediksi (galat kecil/besar)	Tersedia: dua screenshot hasil inferensi pada Gambar 11-12, dengan galat relatif 0,06% dan 52,00%.

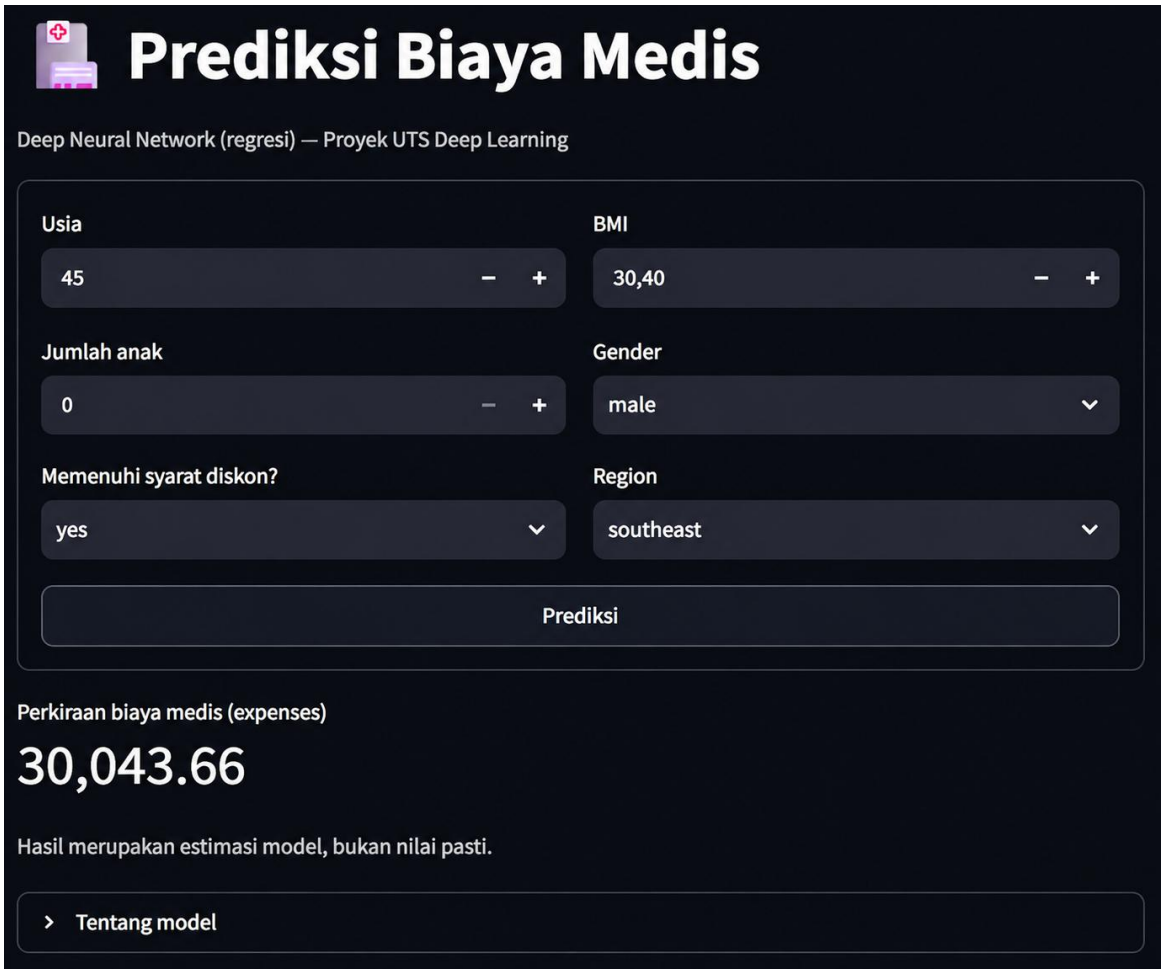
Persyaratan bukti	Status bukti yang tersedia
Video demo dan dokumentasi teknis	Tersedia: video Google Drive (15,51 detik), menampilkan formulir, informasi model, dan dua keluaran prediksi. Bukti dua fitur teknis: Subbab 5.4 dan Lampiran 9.5; uraian *deployment*: Subbab 8.2.
Akses *live* aplikasi	Alamat aplikasi tercantum; hasil pengujian ada pada Gambar 10–12 dan dokumentasi publikasi pada Gambar 13–17.

Bukti tambahan berikut memperlihatkan dua contoh inferensi langsung pada aplikasi Streamlit yang menggunakan model DNN tersimpan. Nilai aktual diperoleh dari data uji proyek; nilai prediksi yang dilaporkan di bawah diambil dari tangkapan layar aplikasi. Karena ini merupakan regresi, contoh dibedakan berdasarkan galat kecil dan galat besar, bukan label klasifikasi benar atau salah.



Gambar 11. Contoh prediksi DNN dengan galat kecil pada aplikasi Streamlit

Kasus A: usia 47 tahun, BMI 26,60, dua anak, perempuan, tidak merokok, wilayah northeast. Nilai aktual 9.715,84 dan prediksi aplikasi 9.721,62; galat absolut 5,78 atau sekitar 0,06% dari nilai aktual.



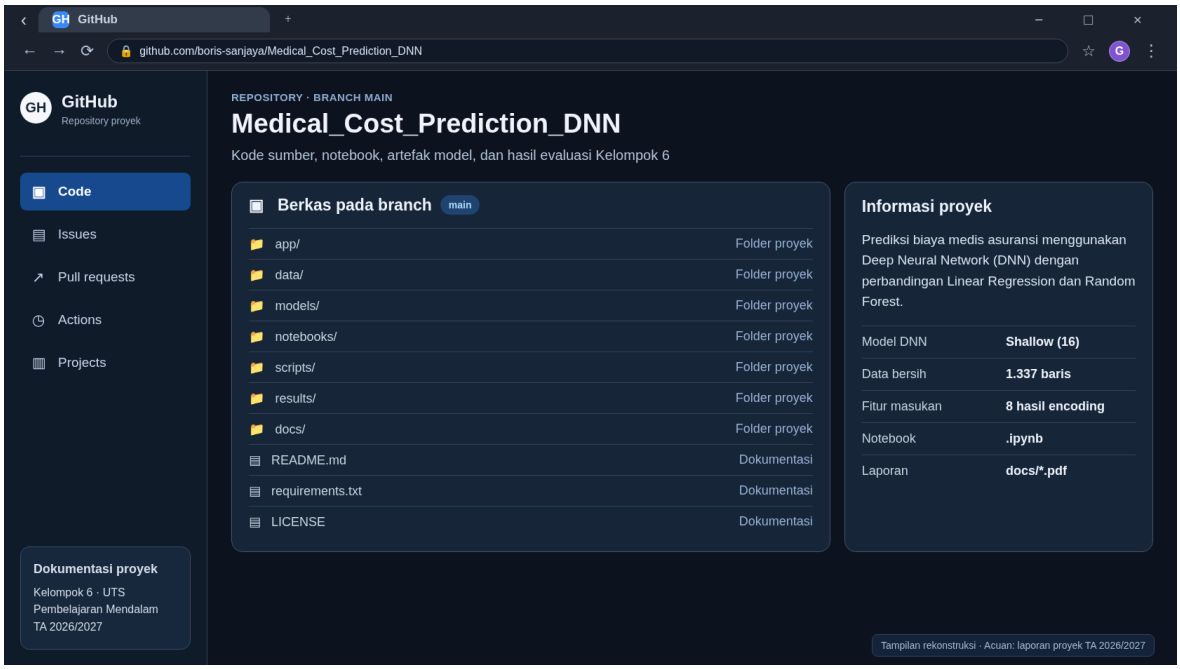
Gambar 12. Contoh prediksi DNN dengan galat besar pada aplikasi Streamlit

Kasus B: usia 45 tahun, BMI 30,40, tanpa anak, laki-laki, merokok, wilayah southeast. Nilai aktual 62.592,87 dan prediksi aplikasi 30.043,66; galat absolut 32.549,21 atau sekitar 52,00% dari nilai aktual.

Interpretasi bukti. Rekaman pada Bagian B menunjukkan interaksi formulir, panel Tentang model, dan dua angka prediksi yang juga dibahas pada Gambar 11–12. Bukti penerapan dua fitur teknis dirujuk melalui Subbab 5.4, Tabel 7–8, kode *notebook*, dan Lampiran 9.5; tahapan penyimpanan model, *inferensi*, dependensi, serta publikasi dijelaskan di Subbab 8.2 dan Gambar 13–15. Penjelasan tertulis tersebut melengkapi isi video, bukan menyatakan bahwa semua tahapan teknis terekam. Label “Memenuhi syarat diskon?” pada rekaman berarti status merokok; kode app/app.py memakai “Status merokok”. Prediksi kasus B dalam CSV adalah 30.043,63, sedangkan layar aplikasi menunjukkan 30.043,66 (selisih 0,03). Nilai aktual berasal dari data uji, bukan hasil *input* formulir.

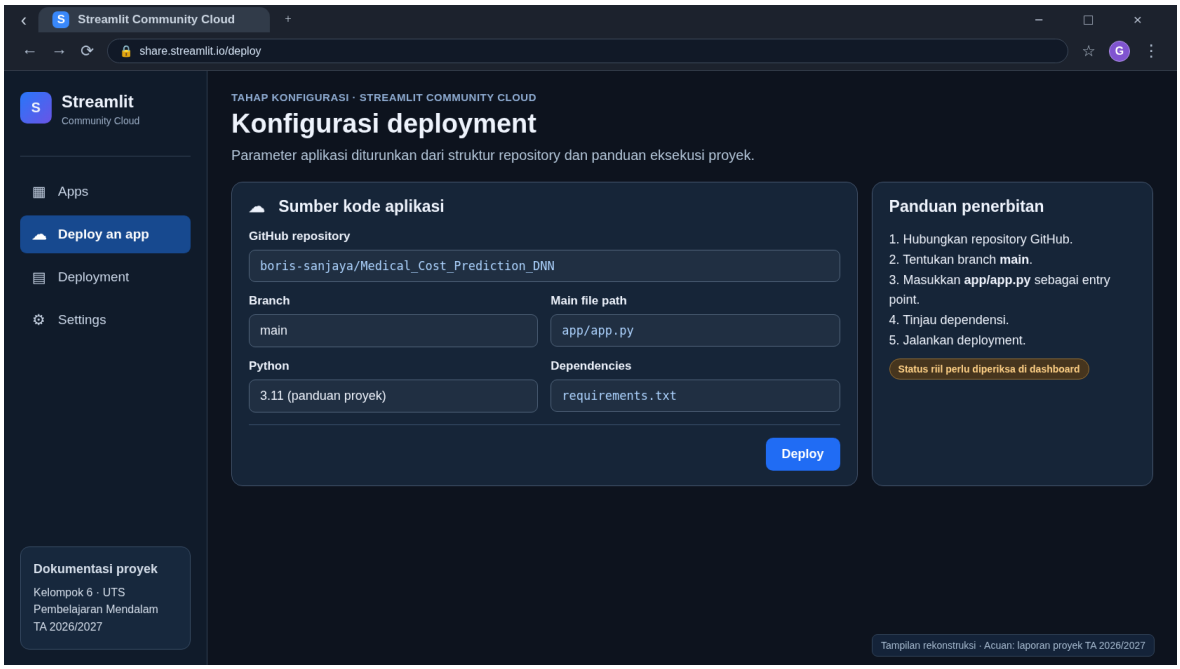
8.5 Dokumentasi visual tahapan publikasi

Gambar 13–17 menyajikan urutan kerja *repository GitHub*, konfigurasi *Streamlit*, pemeriksaan status aplikasi, formulir prediksi, dan informasi model. Gambar tersebut merupakan penyajian visual berdasarkan kode proyek dan hasil evaluasi yang tercatat, bukan rekaman *dashboard* hosting asli. Uraian langkah teknis tersedia pada Subbab 8.2; bukti interaksi aplikasi dalam video dan hasil uji ditunjukkan oleh Gambar 10–12.



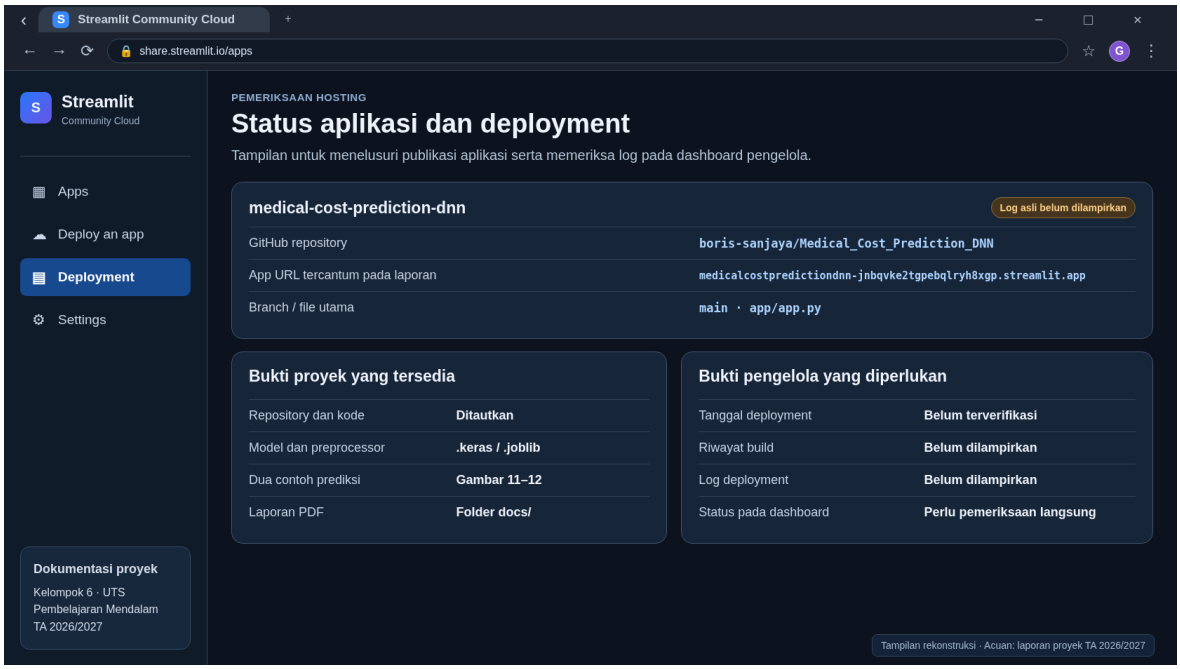
Gambar 13. Tampilan repository GitHub dan struktur berkas proyek

Susunan pada gambar mencakup folder app, data, models, notebooks, scripts, results, dan docs, serta README.md dan requirements.txt. Lokasi rujukan sumber kode adalah repository GitHub yang dicantumkan pada Bagian B.



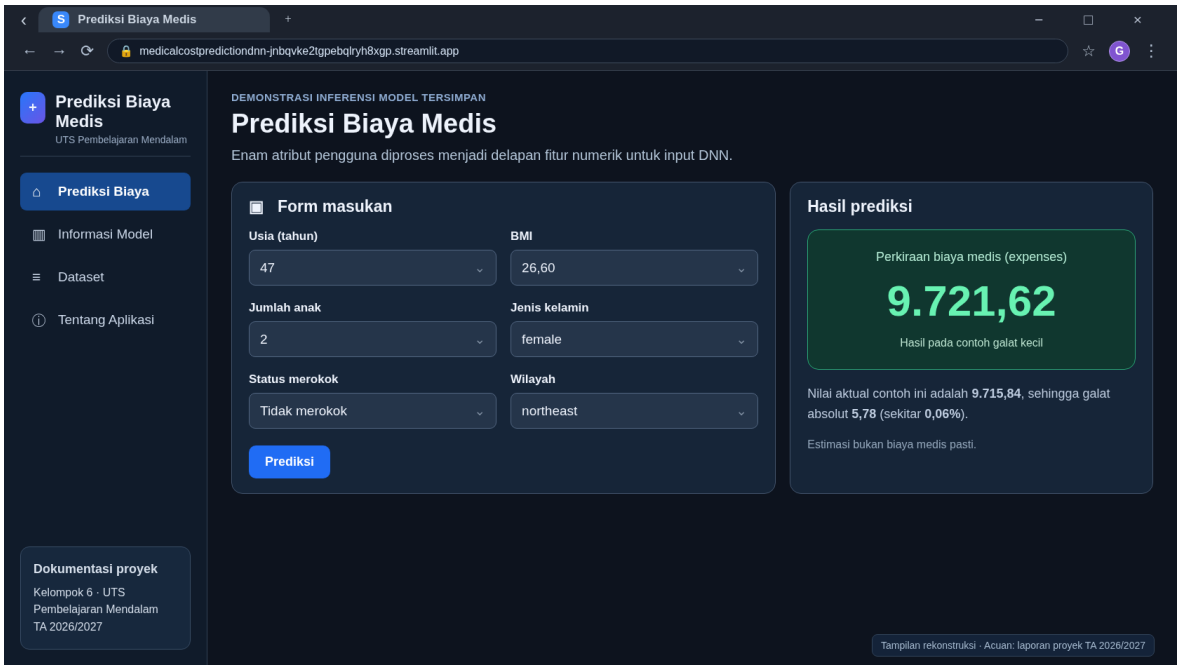
Gambar 14. Tampilan konfigurasi publikasi aplikasi pada Streamlit

Konfigurasi publikasi menggunakan repository boris-sanjaya/Medical_Cost_Prediction_DNN, branch main, dan berkas utama app/app.py. Versi Python pada panduan adalah 3.11; pengaturan sebenarnya tetap perlu diperiksa melalui dashboard hosting.



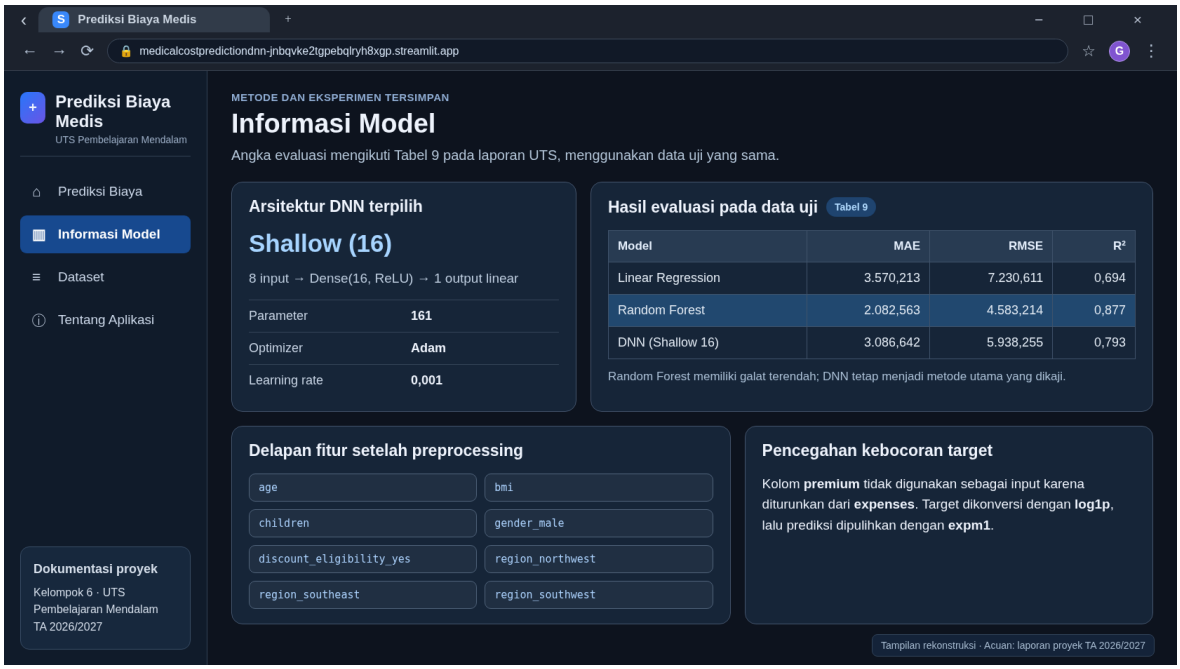
Gambar 15. Tampilan ringkasan status penerbitan dan log aplikasi

Gambar ini menjelaskan komponen yang diperiksa saat publikasi di *Streamlit* Community Cloud, seperti status *build*, dependensi, dan *log* aplikasi. Catatan waktu *deployment* serta *log* server yang sebenarnya hanya dapat dikonfirmasi melalui *dashboard* akun pengelola dan tidak diturunkan dari gambar penyajian alur.



Gambar 16. Tampilan formulir masukan dan hasil estimasi biaya medis

Contoh formulir berisi usia 47 tahun, BMI 26,60, dua anak, perempuan, tidak merokok, dan wilayah northeast. Keluaran 9.721,62 konsisten dengan kasus galat kecil pada Gambar 11; nilai aktual 9.715,84 dan galat absolut 5,78.



Gambar 17. Tampilan halaman informasi metode dan hasil evaluasi

Informasi arsitektur pada gambar mengikuti model DNN terpilih Shallow (16) dengan delapan fitur dan 161 parameter. Tabel evaluasi mengikuti Tabel 9: MAE, RMSE, dan R^2 masing-masing untuk Linear Regression, Random Forest, dan DNN. Gambar merupakan penyajian ulang data laporan, bukan tangkapan layar aktual hasil evaluasi aplikasi.

9. LAMPIRAN

9.1 Pembersihan, *encoding* dan standardisasi

Cuplikan Kode 9.1 - Pembersihan, encoding, dan pembagian data

```
# Notebook: sel 2, 4, dan 11
SEED = 42
for p in ["../data/medical_insurance.csv",
          "data/medical_insurance.csv", "medical_insurance.csv"]:
    if os.path.exists(p):
        DATA_PATH = p
        break

df = pd.read_csv(DATA_PATH)
data = df.drop_duplicates().drop(columns=["premium"]).reset_index(drop=True)
data = pd.get_dummies(
    data, columns=["gender", "discount_eligibility", "region"],
    drop_first=True, dtype=int
)
X = data.drop(columns=["expenses"])
y = data["expenses"]
X_tr, X_tmp, y_tr, y_tmp = train_test_split(
    X, y, test_size=0.30, random_state=SEED
)
X_val, X_te, y_val, y_te = train_test_split(
    X_tmp, y_tmp, test_size=0.50, random_state=SEED
)
num_cols = ["age", "bmi", "children"]
scaler = StandardScaler().fit(X_tr[num_cols])
def prep(Xd):
    Xd = Xd.copy()
    Xd[num_cols] = scaler.transform(Xd[num_cols])
    return Xd
X_tr_s, X_val_s, X_te_s = prep(X_tr), prep(X_val), prep(X_te)
y_tr_log, y_val_log = np.log1p(y_tr), np.log1p(y_val)
```

Penjelasan kode. Cuplikan ini membaca berkas CSV dari beberapa kemungkinan lokasi, menghapus satu baris duplikat, dan membuang *premium* karena nilainya merupakan turunan target *expenses*. Variabel kategorikal (termasuk status merokok yang pada CSV masih bernama discount_eligibility) diubah menjadi delapan fitur numerik, lalu data dipisahkan secara acak dengan *seed* 42 menjadi 935 data latih, 201 validasi, dan 201 uji. *StandardScaler* di-fit pada data latih saja untuk mencegah kebocoran antarbagian data. Target *train* dan *validation* diubah dengan *log1p*; hasil yang ditampilkan sudah berasal dari *notebook* asli.

9.2 Arsitektur, pelatihan dan seleksi DNN

Cuplikan Kode 9.2a - Fungsi arsitektur DNN dan EarlyStopping

```
# Notebook: sel 15 (arsitektur dan callback)
def build_dnn(units=(64, 32, 16), dropout=0.1,
              lr=1e-3, optimizer="adam", activation="relu"):
    tf.keras.backend.clear_session()
    tf.random.set_seed(SEED)
    m = keras.Sequential([layers.Input(shape=(X_tr_s.shape[1],))])
    for u in units:
        m.add(layers.Dense(u, activation=activation))
        if dropout > 0:
            m.add(layers.Dropout(dropout))
    m.add(layers.Dense(1))
    opt = {
        "adam": keras.optimizers.Adam,
        "sgd": keras.optimizers.SGD,
        "rmsprop": keras.optimizers.RMSprop
    }[optimizer](learning_rate=lr)
    m.compile(optimizer=opt, loss="mse", metrics=["mae"])
    return m

def train(model, epochs=800):
    es = keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=25,
        restore_best_weights=True
    )
    return model.fit(
        X_tr_s, y_tr_log,
        validation_data=(X_val_s, y_val_log),
        epochs=epochs, batch_size=32,
        callbacks=[es], verbose=0
    )
```

Penjelasan kode. Fungsi build_dnn() merakit jaringan Dense dan *Dropout* sesuai daftar jumlah neuron, fungsi aktivasi, *optimizer*, serta learning rate. Fungsi *train*() menerapkan *EarlyStopping* pada val_loss dengan patience 25 dan memulihkan bobot terbaik; batas 800 *epoch* adalah batas maksimum, bukan jumlah *epoch* yang pasti dijalankan.

Delapan konfigurasi dan kriteria seleksi (*notebook* sel 18)

```python
# Notebook: sel 18 (delapan konfigurasi yang diuji)
configs = {
    "Shallow (16)": dict(units=(16,), dropout=0.0),
    "2 layer (32-16)": dict(units=(32, 16), dropout=0.0),
    "3 layer (64-32-16)": dict(units=(64, 32, 16), dropout=0.0),
    "3 layer + Dropout 0.1": dict(
        units=(64, 32, 16), dropout=0.1),
    "Deep (128-64-32-16) + D0.2": dict(
        units=(128, 64, 32, 16), dropout=0.2),
    "3 layer + Dropout, SGD": dict(
        units=(64, 32, 16), dropout=0.1,
        optimizer="sgd", lr=1e-2),
    "3 layer + Dropout, RMSprop": dict(
        units=(64, 32, 16), dropout=0.1,
        optimizer="rmsprop"),
    "3 layer + Dropout, tanh": dict(
        units=(64, 32, 16), dropout=0.1,
        activation="tanh"),
}
```

Penjelasan kode. Delapan konfigurasi membandingkan jumlah lapisan, ukuran neuron, *Dropout*, *optimizer*, dan aktivasi. Setiap nama konfigurasi dipakai sebagai kunci eksperimen; Shallow (16) berarti satu lapisan tersembunyi berisi 16 neuron. Variasi ini menilai kecocokan kapasitas *DNN* terhadap data kecil, bukan delapan *dataset* berbeda.

Evaluasi validasi, pemilihan *Shallow* (16), dan pengujian akhir

```
# Notebook: sel 18 dan 20 (validasi, seleksi, uji)
rows = []
for name, cfg in configs.items():
    m = build_dnn(**cfg)
    h = train(m)
    pred = np.expm1(m.predict(X_val_s, verbose=0).ravel())
    r = evaluate(y_val, pred)
    r["Epoch"] = len(h.history["loss"])
    r["Konfigurasi"] = name
    rows.append(r)
exp_df = (pd.DataFrame(rows).set_index("Konfigurasi")
          .sort_values("RMSE"))
best_name = exp_df.index[0]
best_cfg = configs[best_name]
final = build_dnn(**best_cfg)
hist_final = train(final)
pred_te = np.expm1(final.predict(X_te_s, verbose=0).ravel())
results["DNN (terbaik)"] = evaluate(y_te, pred_te)
```

Penjelasan kode. Cuplikan ini mengurutkan delapan model berdasarkan RMSE pada *validation set*, memilih konfigurasi *Shallow* (16), lalu melatih model akhir dan menghitung metrik pada *test set*. Pemilihan konfigurasi dilakukan menggunakan data validasi, bukan data uji. Angka evaluasi di laporan berasal dari output tersimpan; cuplikan ini bukan instruksi untuk menjalankan ulang *notebook*.

9.3 Penyimpanan model dan inferensi aplikasi

Cuplikan Kode 9.3a - Penyimpanan model dan preprocessor

```
# Notebook: sel 23 (penyimpanan artefak)
os.makedirs("../models", exist_ok=True)
final.save("../models/dnn_medical_cost.keras")
joblib.dump(
    {"scaler": scaler, "num_cols": num_cols,
     "columns": list(X.columns)},
    "../models/preprocessor.joblib"
)
```

Penjelasan kode. *Model DNN* akhir disimpan sebagai dnn_medical_cost.*keras*. Berkas preprocessor.*joblib* menyimpan objek *scaler*, daftar kolom numerik, dan urutan seluruh fitur saat pelatihan, agar aplikasi memakai transformasi masukan yang sama dengan *notebook*.

Memuat model tersimpan dan menghitung prediksi (app/app.py)

```python
# app/app.py (versi revisi): inferensi tanpa pelatihan ulang
BASE = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE, "..", "models", "dnn_medical_cost.keras")
PREP_PATH = os.path.join(BASE, "..", "models", "preprocessor.joblib")

@st.cache_resource
def load_artifacts():
    model = keras.models.load_model(MODEL_PATH)
    prep = joblib.load(PREP_PATH)
    return model, prep

def predict(model, prep, age, gender, bmi,
            children, smoking_status, region):
    row = {c: 0 for c in prep["columns"]}
    row.update({"age": age, "bmi": bmi, "children": children})
    if gender == "male" and "gender_male" in row:
        row["gender_male"] = 1
    # Nama kolom lama dipertahankan agar model tetap kompatibel.
    if smoking_status == "yes" and "discount_eligibility_yes" in row:
        row["discount_eligibility_yes"] = 1
    key = f"region_{region}"
    if key in row:
        row[key] = 1
    X = pd.DataFrame([row])[prep["columns"]].astype(float)
    X[prep["num_cols"]] = prep["scaler"].transform(X[prep["num_cols"]])
    log_pred = model.predict(X, verbose=0).ravel()[0]
    return float(np.expm1(log_pred))
```

Penjelasan kode. load_artifacts() membaca model dan *preprocessor* dari folder models. Fungsi predict() mengubah Status merokok menjadi fitur internal discount_eligibility_yes, membentuk delapan kolom dalam urutan pelatihan, mengisi fitur kategorikal, menstandardisasi age, bmi, dan children, lalu menjalankan model.predict(). Keluaran target pada skala *log* dipulihkan dengan expm1. Proses ini adalah inferensi, bukan pelatihan ulang.

Form enam atribut dan keluaran prediksi (app/app.py)

```python
# app/app.py (versi revisi): label sesuai makna data sumber
with st.form("input_form"):
    c1, c2 = st.columns(2)
    age = c1.number_input(
        "Usia", min_value=18, max_value=100, value=30, step=1)
    bmi = c2.number_input(
        "BMI", min_value=10.0, max_value=60.0, value=27.0, step=0.1)
    children = c1.number_input(
        "Jumlah anak", min_value=0, max_value=10, value=0, step=1)
    gender = c2.selectbox("Gender", ["female", "male"])
    smoking_status = c1.selectbox(
        "Status merokok", ["no", "yes"],
        format_func=lambda v: "Tidak merokok" if v == "no" else "Merokok")
    region = c2.selectbox("Region", [
        "northeast", "northwest", "southeast", "southwest"])
    submitted = st.form_submit_button(
        "Prediksi", use_container_width=True)
if submitted:
    result = predict(model, prep, age, gender, bmi,
                     children, smoking_status, region)
    st.metric("Perkiraan biaya medis (expenses)", f"{result:,.2f}")
```

Penjelasan kode. Formulir Streamlit menerima enam atribut, termasuk Status merokok dengan pilihan Merokok dan Tidak merokok. Kedua pilihan dipetakan menjadi yes/no sesuai fitur internal model tersimpan. Setelah tombol Prediksi ditekan, aplikasi memanggil predict() dan menampilkan estimasi expenses melalui st.metric(). Proses inferensi menggunakan bobot DNN dan *preprocessor* yang sudah tersimpan.

9.4 Evaluasi, visualisasi dan konsistensi hasil

Fungsi evaluasi dan model pembanding (*notebook* sel 13)

```
# Notebook: sel 13 (evaluasi dan baseline)
def evaluate(y_true, y_pred):
    return {
        "MAE": mean_absolute_error(y_true, y_pred),
        "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
        "R2": r2_score(y_true, y_pred)
    }
results = {}
lr = LinearRegression().fit(X_tr_s, y_tr_log)
results["Linear Regression"] = evaluate(
    y_te, np.expm1(lr.predict(X_te_s)))
rf = RandomForestRegressor(
    n_estimators=300, min_samples_leaf=3,
    random_state=SEED, n_jobs=-1
).fit(X_tr_s, y_tr_log)
results["Random Forest"] = evaluate(
    y_te, np.expm1(rf.predict(X_te_s)))
```

Penjelasan kode. Fungsi evaluate() menghitung *MAE*, *RMSE*, dan R^2 pada skala *expenses*. *Linear Regression* dan *Random Forest* merupakan *baseline* untuk menilai posisi *DNN*. Keduanya memakai split dan transformasi yang sama; prediksi dalam skala log dipulihkan dengan *expm1* sebelum menghitung metrik.

Kurva *loss*, prediksi versus aktual, dan residual (sel 16 dan 21)

```
# Notebook: sel 16 dan 21 (grafik dari hasil tersimpan)
plt.figure(figsize=(7, 4))
plt.plot(history.history["loss"], label="Train loss")
plt.plot(history.history["val_loss"], label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("MSE (log target)")
plt.title("Kurva Loss DNN")
plt.legend()
plt.show()

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
ax[0].scatter(y_te, pred_te, alpha=.6)
lim = [0, max(y_te.max(), pred_te.max())]
ax[0].plot(lim, lim, "r--")
ax[0].set_xlabel("Aktual")
ax[0].set_ylabel("Prediksi")
ax[0].set_title("Prediksi vs Aktual (test)")
sns.histplot(y_te.values - pred_te, kde=True, ax=ax[1])
ax[1].set_title("Distribusi residual (aktual - prediksi)")
plt.tight_layout()
plt.show()
```

Penjelasan kode. Cuplikan visualisasi pertama membentuk kurva *loss* dan *val_loss* model DNN awal; cuplikan kedua menggambar prediksi terhadap nilai aktual serta histogram *residual* pada data uji. Grafik *loss* tidak boleh diberi label sebagai kurva model akhir *Shallow* (16). Seluruh hasil tabel utama tetap merujuk pada output *notebook* yang telah disimpan.

9.5 Analisis tambahan dampak data leakage (ablasi independen)

Tujuan. Mengukur perubahan MAE, RMSE, dan R^2 ketika kolom *premium* yang merupakan turunan target *expenses* dimasukkan sebagai fitur. Ini eksperimen diagnostik terpisah, bukan eksekusi ulang *notebook* maupun pelatihan DNN.

Metode. Setelah satu duplikat dihapus, terdapat 1.337 baris. Kategori dienkode dengan *drop_first*=True; pembagian seed=42 ialah 935 *train*, 201 *validation*, 201 *test*, identik dengan *notebook*. *Scaler* hanya di-fit pada *train*; model mempelajari *log1p*(*expenses*) dan prediksi diubah menggunakan *expm1*. Kondisi tanpa dan dengan *premium* memakai indeks data yang sama. Model: *Linear Regression* dan *Random Forest* (300 pohon, min_samples_leaf=3, seed=42).

Tabel 12. Perbandingan ablasi dengan dan tanpa kolom premium (data uji)

Model	Fitur premium	MAE	RMSE	R^2
Linear Regression	Dihapus (aman)	3.570,21	7.230,61	0,694
Linear Regression	Dipakai (bocor)	4.239,16	11.865,72	0,175
Random Forest	Dihapus (aman)	2.082,56	4.583,21	0,877
Random Forest	Dipakai (bocor)	359,00	1.523,66	0,986

Interpretasi. Pada *Random Forest*, penggunaan *premium* menurunkan MAE dari 2.082,56 menjadi 359,00 (82,76%) dan RMSE dari 4.583,21 menjadi 1.523,66 (66,76%). Angka yang terlihat lebih baik itu tidak sah sebagai pengukuran kemampuan prediksi karena kebocoran target. Sebaliknya, pada *Linear Regression*, MAE meningkat dari 3.570,21 menjadi 4.239,16 dan RMSE meningkat dari 7.230,61 menjadi 11.865,72. Jadi, fitur bocor tidak selalu menurunkan error. Metrik tanpa *premium* cocok dengan *baseline notebook* asli.

Batas interpretasi. Eksperimen ablasi menggunakan scikit-learn dengan dua model *baseline* yang dijalankan secara terpisah dari pelatihan DNN. *Notebook* utama, model .keras, dan hasil evaluasi DNN tetap dipertahankan. Skrip tersedia pada scripts/Analisis_Ablasi_Data_Leakage_Kelompok_6.py dan keluarannya pada results/Hasil_Ablasi_Kelompok_6.csv. Percobaan ini tidak menggantikan bukti proses *deployment* atau catatan asal pengunduhan *dataset* oleh anggota.

Cuplikan 9.5a-9.5c menjelaskan tiga tahap dari skrip yang sama: pembacaan data dan pembagian skenario, standardisasi per skenario, serta evaluasi dua *baseline*. Kode pada gambar telah dipenggal ulang pada beberapa baris dan diberi komentar konteks agar terbaca pada kertas A4; logika yang digunakan sesuai dengan skrip dalam folder scripts/, sedangkan hasil perhitungannya ada di results/. Prosedur ini tidak melatih ulang model DNN.

Cuplikan Kode 9.5a - Dataset proyek, pembersihan, dan pemisahan

```
# Sumber: scripts/Analisis_Ablasi_Data_Leakage_Kelompok_6.py
parser = argparse.ArgumentParser(
    description="Ablasi dampak kebocoran fitur premium (dua baseline)"
)
root = Path(__file__).resolve().parents[1]
parser.add_argument(
    '--data', type=Path,
    default=root / 'data/medical_insurance.csv'
)
parser.add_argument(
    '--out', type=Path, default=root / 'results'
)
args = parser.parse_args()
args.out.mkdir(parents=True, exist_ok=True)
# Jalur masukan utama dari data repository
raw = pd.read_csv(args.data)
base = pd.get_dummies(
    raw.drop_duplicates().reset_index(drop=True),
    columns=['gender', 'discount_eligibility', 'region'],
    drop_first=True, dtype=int
)
y = base['expenses']
safe = base.drop(columns=['expenses', 'premium'])
leak = base.drop(columns=['expenses'])
```

Penjelasan kode. Skrip menggunakan data/medical_insurance.csv pada direktori proyek hasil kloning repository GitHub. Cuplikan memperlihatkan alur utama pembacaan data, pembersihan, dan pembentukan dua skenario fitur. Setelah satu baris duplikat dihapus, variabel kategorikal diubah melalui *one-hot encoding*. Skenario aman menghapus premium, sedangkan skenario pembanding menyertakannya untuk mengukur pengaruh kebocoran target. Indeks data latih, validasi, dan uji dibentuk sekali menggunakan random_state=42 agar kedua kondisi menggunakan observasi yang sama.

Cuplikan Kode 9.5b - Skenario aman/bocor dan standardisasi

```
# Analisis_Ablasi_Data_Leakage_Kelompok_6.py
rows = []
for label, X, numeric in [
    ('Tanpa premium (aman)', safe, ['age', 'bmi', 'children']),
    ('Dengan premium (bocor)', leak,
     ['age', 'bmi', 'children', 'premium'])
]:
    Xtr, Xte = X.iloc[itr].copy(), X.iloc[ite].copy()
    scaler = StandardScaler().fit(Xtr[numeric])
    Xtr[numeric] = scaler.transform(Xtr[numeric])
    Xte[numeric] = scaler.transform(Xte[numeric])
    ytr = np.log1p(y.iloc[itr])
    yte = y.iloc[ite]
    # Latih dua baseline pada Xtr; evaluasi pada Xte.
```

Penjelasan kode. Pada setiap kondisi, *StandardScaler* hanya di-fit pada subset *train*. Kolom *premium* ikut distandardisasi hanya pada kondisi bocor, sedangkan target pelatihan diubah memakai *log1p*. Struktur ini mengisolasi perbedaan utama pada keberadaan *premium*, bukan pada pembagian data.

Cuplikan Kode 9.5c - Model pembanding, evaluasi, dan ekspor hasil

```python
# Analisis_Ablasi_Data_Leakage_Kelompok_6.py
methods = {
    'Linear Regression': LinearRegression(),
    'Random Forest': RandomForestRegressor(
        n_estimators=300, min_samples_leaf=3,
        random_state=42, n_jobs=-1)
}
for name, model in methods.items():
    model.fit(Xtr, ytr)
    pr = np.expm1(model.predict(Xte))
    rows.append({
        'Kondisi': label, 'Model': name,
        'MAE': mean_absolute_error(yte, pr),
        'RMSE': np.sqrt(mean_squared_error(yte, pr)),
        'R2': r2_score(yte, pr)
    })
res = pd.DataFrame(rows)
res.to_csv(args.out / 'Hasil_Ablasi_Kelompok_6.csv', index=False)
```

Penjelasan kode. Dua *baseline* dilatih terpisah untuk setiap kondisi, kemudian diprediksi pada *test* set yang sama. Nilai prediksi dikembalikan ke satuan *expenses* dengan *expm1* sebelum *MAE*, *RMSE*, dan R^2 dihitung. Empat baris hasil disimpan dalam *Hasil_Ablasi_Kelompok_6.csv* sebagai dasar Tabel 12; hasil *DNN* asli tidak dihitung ulang.

10. DAFTAR PUSTAKA

[1] mirichoi0218, "*Medical Cost Personal Datasets*," Kaggle, https://www.kaggle.com/datasets/mirichoi0218/insurance, diakses 8 November 2026. Sumber publik yang memiliki skema asli tujuh atribut.

[2] Kelompok 6, "Medical_Cost_Prediction_DNN," *repository* GitHub, github.com/boris-sanjaya/Medical_Cost_Prediction_DNN, diakses 7 November 2026.

[3] TensorFlow, "Keras model *training* APIs," dokumentasi, tensorflow.org/api_docs/python/tf/keras/Model.

[4] scikit-learn, "Model evaluation: quantifying the quality of predictions," dokumentasi, scikit-learn.org/stable/modules/model_evaluation.html.

[5] Streamlit, "Streamlit documentation," docs.streamlit.io.

[6] Kelompok 6, "medical_cost_dnn.ipynb" dan "medical_insurance.csv," berkas pada folder notebooks/ dan data/ di repository GitHub Medical_Cost_Prediction_DNN, 2026.

[7] adrian-io, "*insurance.csv*," medical-cost-*prediction*, GitHub, https://github.com/adrian-io/medical-cost-prediction/blob/main/insurance.csv, diakses 8 November 2026. Pembanding untuk verifikasi seluruh 1.338 baris setelah penamaan ulang dan pembulatan.

BAGIAN B. TAUTAN

TAUTAN PROYEK

Bagian ini menyediakan tautan *GitHub*, video *MP4* demonstrasi aplikasi berdurasi 15,51 detik, dan aplikasi *Streamlit*. Video mendokumentasikan pengisian formulir, panel informasi model, dan dua keluaran prediksi. Penjelasan serta bukti dua fitur teknis berada pada Subbab 5.4, Tabel 7–8, dan Lampiran 9.5; alur *deployment* dibahas pada Subbab 8.2 dan Gambar 13–17. Uraian tertulis tersebut melengkapi rekaman singkat tanpa menambahkan adegan yang tidak diperlihatkan. Seluruhnya merujuk pada *dataset*, model tersimpan, dan evaluasi yang sama.

1. *Source code* (GitHub)

https://github.com/boris-sanjaya/Medical_Cost_Prediction_DNN

2. Video demo dan tahapan *deployment*

https://drive.google.com/file/d/17TalNHyt0PP1aYiI1nqoZFQkRoaSFuSp/view?usp=sharing

3. Aplikasi yang sudah di-deploy

https://medicalcostpredictiondnn-jnbqvke2tgpebqlryh8xgp.streamlit.app/